FlacToAiff – Code Review & Roadmap

Oct 25, 2026 · @Louis Schumann

Summary

The code is in good shape for a 2,760-line app: careful file handling, real verification and an end-to-end test suite most paid converters don't have. Nothing in it blocks the three goals, but one thing has to happen first: the core is written around FLAC in about a dozen places, so WAV and ALAC need a small refactor before they are added, not bolted on.

Recommended order:

1. **Intel + a signed, notarized release.** Mostly build work, almost no code. This is what makes the app usable for anyone but you.
2. **Source-format refactor**, then **WAV**, then **ALAC**. WAV is easy and can be verified as strictly as FLAC. ALAC can't, and needs a lossy-AAC guard.
3. **rekordbox**, in two stages: target-player presets and a USB check first, then playlist conversion through rekordbox XML. Editing rekordbox's own database is not recommended for a product you sell.

Reviewed: `master` at commit 2046626 (40 commits), all files in `Sources/`, `build.sh`, `test.sh`, and the plans in `docs/`.

Code review

No correctness bugs that lose audio or overwrite files. The findings below are about what the roadmap will run into, plus a few small tag issues.

What's strong

- **Publishing is genuinely safe.** Hidden temp file, then `renamex_np(RENAME_EXCL)`, with `link()` and a check-then-rename fallback for exFAT/FAT32 (`Converter.publish`). Stale partials are cleaned by checking whether the writing process still exists.
- **Verification is honest.** The MD5 check against STREAMINFO is independent of Apple's decoder, so it proves the AIFF is the FLAC. It streams through two 1 MiB buffers, so memory stays flat. Labels never claim more than was checked.
- **Quitting is handled.** `InFlight` kills running `afconvert` processes and removes their temp files.
- **The watch folder is thought through:** file IDs instead of paths, Date Added of parent folders, settle times, catch-up after relaunch, drives that mount later.

- **Tests are end-to-end on real files**, including truncated and tampered files, exFAT and races, with test-only hooks compiled out of the app.
- **No dependencies.** Fewer things to break, nothing to license.

Findings

#	Where	Finding	Severity	Fix
1	`Converter`, `Jobs`, `FolderWatcher`, `Verifier`, UI strings	FLAC is hard-wired: `convert(flacURL:)`, `ConvertError.notFlac`, `.flac` filters in `Jobs.flacs` and `FolderWatcher.noticed`, STREAMINFO-shaped verification, "Drop .flac files" text	High for the roadmap	Introduce a source abstraction before adding formats (Step 2)
2	`Converter.convert`	"Already converted" means only "`<name>.aiff` exists". With several input formats, `track.flac` and `track.wav` in one folder collide, and the second is wrongly skipped	High once WAV/ALAC exist	Write a source fingerprint into the AIFF (TXXX frame) and compare it on skip
3	`build.sh`, `test.sh` line 18	`arm64-apple-macos12.0` hard-coded in both	Blocks Intel	Build both slices and `lipo` them (Step 1)
4	`build.sh`	Bundle ID `local.flactoaiff`, version 1.0 fixed in the script	Blocks release	Reverse-DNS ID you own, version from one place, migrate `UserDefaults`
5	`Settings`	Folders stored as plain paths. A renamed USB stick or moved folder breaks Save to and Watch	Medium	Store bookmark data; required anyway if you ever go to the Mac App Store
6	`ConversionQueue`	One conversion at a time. `afconvert` is single-threaded, so a 300-track batch uses one core	Medium	2–4 parallel jobs to local disks, 1 to USB sticks; temp names are already collision-free
7	`Verifier.verifyTags`	One tag that doesn't read back blocks the whole file. Strict is right for trust, but a single odd TXXX value makes a track unconvertible	Medium (product call)	Keep strict for named frames; for custom TXXX fields, drop the field and say so in the row

#	Where	Finding	Severity	Fix
8	`TagIO.id3Frames`	`TYER` = first 4 characters of `DATE`. A date like `03.05.2026` becomes `03.0`	Low	Take the first 4-digit run, or skip `TYER`
9	`TagIO.id3Frames`	Every PICTURE block becomes an APIC, in file order, including icons (types 1, 2) and URL pictures (mime `-->`). Players that take the first APIC can show the wrong image	Low	Front cover (type 3) first; skip icons and URL pictures
10	`main.swift`	480 lines mixing CLI, view and app delegate	Low now, Medium once rekordbox UI arrives	Split into `CLI.swift`, `DropView.swift`, `AppDelegate.swift`
11	`ConversionQueue.rows`	Grows without limit while a watch folder runs for days	Low	Cap at a few thousand rows

The earlier review in `docs/CODE_REVIEW-2026-07-09.md` (bit-depth detection, pipe deadlocks, partial files) is fully addressed in the current code.

Step 1 — Intel, and a release people can open

The Swift code is already architecture-neutral: no SIMD, no inline assembly, both CPUs are little-endian and 64-bit, and `afconvert` / `afinfo` exist on Intel Macs. Intel support is a build and test change, so pair it with signing and notarization and ship one proper release.

1a. Universal binary

1. In `build.sh`, compile twice, `-target arm64-apple-macos12.0` and `-target x86_64-apple-macos12.0`, into the stage folder.
2. Merge with `lipo -create -output …/FlacToAiff arm64-bin x86_64-bin`.
3. Run the `minos` check per slice: `otool -l -arch arm64` and `otool -l -arch x86_64`.
4. In `test.sh`, build the `TEST_HOOKS` binary the same way, instead of the hard-coded `arm64` target on line 18.
5. Run the whole suite twice on your Mac: natively, then under Rosetta with `arch -x86_64 ./test.sh` (install Rosetta first if needed).
6. Before the release, run it once on a real Intel Mac. Rosetta is close, but not proof.

1b. Decide the minimum macOS

The Intel Macs DJs still carry are often old. A 2013–2014 MacBook Pro stops at macOS 11 Big Sur; a 2015 one at macOS 12. The code uses nothing newer than macOS 11 as far as I can see (SwiftUI `Menu`, `LazyVStack`, `onDrop` with `UTType`, CryptoKit). Try `-target …-macos11.0`; if it compiles and the tests pass on an 11 machine, you reach a noticeably bigger group of old laptops. Keep in mind Apple has said macOS 26 is the last release for Intel Macs, so the Intel audience will shrink from here.

1c. Signing, notarization, packaging

1. Join the Apple Developer Program (99 USD per year) and create a **Developer ID Application** certificate.
2. Change the bundle ID from `local.flactoaiff` to a reverse-DNS name you own, and read the old `UserDefaults` domain once so existing settings survive.
3. Sign with the hardened runtime: `codesign --force --options runtime --timestamp --sign "Developer ID Application: …"`. Launching `/usr/bin/afconvert` works under the hardened runtime without extra entitlements.
4. Notarize with `xcrun notarytool submit … --wait`, then `xcrun stapler staple`. `notarytool` ships with Xcode; check whether your Command Line Tools have it before planning around "no Xcode".
5. Ship a `.dmg` (or zip) on GitHub Releases. Add a `release.sh` so all of this is one command.
6. Remove "Apple Silicon only, build it yourself" from the README and the quarantine workaround from Troubleshooting.

Done when: a downloaded build opens with a double-click on an Intel and an Apple Silicon Mac, and `test.sh` passes on both architectures.

Step 2 — WAV and ALAC input

`afconvert` already decodes WAV and ALAC, so the conversion itself is free. The work is in reading each format's tags, verifying honestly, and keeping "Already converted" correct once several formats can produce the same AIFF name.

2a. Refactor first, with no change in behaviour

Pull everything FLAC-specific behind one small interface, so each format is one new file:

```
protocol AudioSource {
    static var extensions: Set<String> { get }      // ["flac"], ["wav",
"wave"], ["m4a"]
    init?(url: URL)                                 // nil = not this format,
or unreadable
    var format: SourceFormat { get }                // depth, rate, channels,
frames, isFloat, codec
    func tags() -> Tags                             // the existing Tags
model
    func audioFingerprint() -> Data?                // MD5 in FLAC's
canonical form, or nil
}
```

- `FlacSource` wraps today's `TagIO.flacStreamInfo` and `readFlacTags`. Its fingerprint is the STREAMINFO MD5.
- `Converter.convert(source:destDir:options:)` replaces `convert(flacURL:…)`. `Verifier` compares the AIFF against `audioFingerprint()` when there is one.
- `Jobs.flacs` becomes `Jobs.audioFiles(in:)`, driven by the registered extensions; `FolderWatcher` uses the same list.
- **Gate:** `test.sh` passes unchanged. Only then add formats.

In the same step, fix finding 2: write a `TXXX:FLACTOAIFF_SOURCE` frame holding the source's fingerprint (or size + modification date when there is none). On "exists", read it back: same source → skip; different source → write `track (wav).aiff` or report a name clash, your call.

2b. WAV (easy, and as strict as FLAC)

- **Verification:** a 16/24/32-bit integer WAV stores samples as signed little-endian, which is exactly the form your `audioMD5` already builds. The MD5 of the WAV's `data` chunk is therefore a decoder-independent fingerprint, so WAV gets the full `verified` label.
- **8-bit WAV** is unsigned; flip the top bit before hashing.
- **32-bit float WAV** (common from DAWs) can't become a lossless integer AIFF, and float AIFF won't play on CDJs. Convert to 24-bit with dither and label it as reduced, or skip it with a clear message.
- **Header variants to parse:** `WAVE_FORMAT_EXTENSIBLE` (format in the sub-format GUID), odd-length chunks with pad bytes, `data` sizes of 0 or `0xFFFFFFFF` from streaming writers, and RF64 for files over 4 GB (rare for tracks; reject cleanly if `afconvert` can't read it).

- **Tags:** two places. `LIST/INFO` (INAM title, IART artist, IPRD album, IGNR genre, ICRD date, ITRK track, ICMT comment) and an embedded ID3 chunk (`id3` or `ID3`), which store WAVs often carry. The ID3 one can be v2.4, so you need an ID3 reader that maps v2.3/v2.4 frames into `Tags` . Write it once; it also covers FLACs with an ID3 header.
- **Watch folder:** many writers fill in the WAV header sizes last. Treat a WAV as settled only when the `data` size matches the file size.

2c. ALAC (harder to verify)

- **Check the codec, not the extension.** `.m4a` is just as often lossy AAC. Read the codec (`alac` vs `aac`) with `afinfo -x` or `AVAudioFile` , and skip AAC with a message: converting it makes a file 5–10× larger with no gain, and CDJs play AAC anyway. Skip `.m4p` (DRM).
- **Tags** come easily through AVFoundation's iTunes metadata: `©nam` , `©ART` , `©alb` , `aART` , `©gen` , `©wrt` , `©day` , `trkn` / `disk` (number pairs), `tmpo` (integer BPM), `©cmt` , `cprt` , `covr` , and free-form `----:com.apple.iTunes:…` atoms such as `INITIALKEY` .
- **Verification:** ALAC stores no checksum. Decoding again with `AVAudioFile` and comparing uses the same Apple decoder as `afconvert` , so it proves the AIFF was written intact, not that the decode is right. Keep your rule that a label never claims more than was checked: length + format + tags, labelled `tags verified` , or a new, clearly worded label for the decode comparison.
- **Bit depths:** ALAC can be 16, 20, 24 or 32-bit; reuse the existing depth mapping.

2d. UI and tests

- Wording: "Drop audio files or folders", "No .flac files found" → "No FLAC, WAV or ALAC files found".
- Add `.wav` and `.m4a` to `CFBundleDocumentTypes` (rank `None` , as today).
- Test inputs need no Python: `afconvert -f WAVE -d LEI24` , `-d LEF32` (float), `-f m4af -d alac` , and `-f m4af -d aac` for the AAC guard.
- New checks: WAV `verified` , float WAV handled, WAV with INFO and ID3 tags, ALAC tags + artwork, AAC skipped, `x.flac` + `x.wav` in one folder.

Done when: a mixed folder of FLAC, WAV and ALAC converts with correct labels, and nothing is wrongly reported as "Already converted".

Step 3 — rekordbox focus

The real pain isn't converting files, it's that rekordbox treats a converted file as a new track and loses its cues. Build towards that in three stages, from no risk to some risk, and stop before touching rekordbox's own database.

Two open-source tools already work here, both command-line and aimed at technical users: rekordbox-playlist-converter (Python + ffmpeg, GPL-3.0, reads a rekordbox XML, writes a new XML for import, presets for 19 CDJ/XDJ models) and rekordbox-edit (edits the database directly). Your angle is a native app with nothing to install, and verified output.

3a. Target-player presets (no rekordbox access at all)

- Replace "Original / CDJ / CD" with a **target** menu: CDJ-3000, CDJ-2000NXS2, CDJ-2000NXS, common XDJs, audio CD, Apple Music. Each target maps to limits (formats, max rate, max depth) that drive today's `OutputMode` logic.
- Keep the table of limits in one data file, with the manual page each value comes from. Wrong data here is the fastest way to lose a DJ's trust.
- Add "only convert what the target can't play": a 44.1 kHz FLAC aimed at a CDJ-3000 is left alone.

3b. USB check (read-only)

- Pick a USB stick and a target; the app scans its audio files and lists what won't play: FLAC on an NXS, 96 kHz on a 48 kHz deck, 32-bit, AAC on an older model.
- Read-only, so it can't break anything. It answers the question DJs ask before every gig in an unknown booth.
- Don't offer to fix files on the stick: replacing them breaks rekordbox's export. Point to the source tracks instead.

3c. Playlist conversion through rekordbox XML

rekordbox can export its collection as XML (File → Export Collection in xml format) and show an imported XML in its tree. The XML holds each track's file location, beatgrid (`TEMPO`) and cue points (`POSITION_MARK`), in seconds.

1. The user exports the XML and drops it on the app.
2. The app lists playlists; the user picks one and a target.
3. Tracks the target can't play are converted, with all existing verification.

4. The app writes a new XML: same track attributes, `Location` pointing to the AIFF, `TEMPO` and `POSITION_MARK` copied. Lossless conversions keep every sample in place, so cue times stay exact. For resampled tracks, check that the resampler adds no delay before trusting the cues.
5. The user imports that XML in rekordbox.

To find out by testing before you promise anything:

- Does importing create duplicates (FLAC and AIFF both in the collection), or can the existing track be updated? The competing tool assigns new track IDs, which suggests duplicates.
- Do hot cues, memory cues, loops, colours, comments and ratings all survive? My Tags, play history and waveforms likely don't; rekordbox re-analyses.
- `Location` is a percent-encoded `file://localhost/` URL. Test non-Latin names, and libraries exported from Windows.
- Compare with rekordbox's own Relocate on missing files: if converting into the same folder and relocating keeps everything, a guided relocate may beat XML.

3d. Direct database editing — not recommended

rekordbox 6/7 keep the library in an encrypted SQLite database. Tools open it with a key found by reverse engineering. For a paid product that is the wrong risk: one rekordbox update can break it, a bug can corrupt a DJ's whole library, and you would own that support case. If you ever do it, back up the database first, refuse while rekordbox is running, and make it opt-in.

Done when: a playlist with FLAC and hi-res tracks comes back into rekordbox as AIFF with hot cues and beatgrid intact, tested on rekordbox 7 and on at least two CDJ models.

What to consider

Order of work

Each step ends with `./build.sh && ./test.sh` green and a release, so the app is always shippable. Effort is a rough guess in focused working days.

1. **Release housekeeping** — bundle ID, version, licence decision, README (1 day).
2. **Step 1:** universal binary, minimum macOS, signing, notarization, `release.sh` (2–3 days, plus one session on an Intel Mac).
3. **Hand it to 10–20 DJs** in your network and collect what breaks. This is the cheapest market test you'll get.
4. **Step 2a:** source refactor + fingerprint for "Already converted" (2–3 days).

5. **Step 2b:** WAV (2–3 days). **Step 2c:** ALAC (2 days).
6. **Step 3a + 3b:** target presets and USB check (3–4 days, most of it collecting player limits from manuals).
7. **Step 3c:** XML playlist conversion (1–2 weeks, mostly testing in rekordbox).

Testing on real gear

Apple's tag reader passing is not the same as a CDJ showing the right title and artwork. Your venue contacts are the one test lab competitors don't have. Keep one test stick with a fixed set of tracks (16/24-bit, 44.1/48/96 kHz, non-Latin tags, large artwork) and note per player what shows and plays:

Player / software	Plays	Title, artist, key, BPM	Artwork	Cues after XML import
rekordbox 7 (Mac)				
CDJ-3000				
CDJ-2000NXS2				
CDJ-2000NXS				
XDJ (most common in your venues)				
Apple Music				

Product decisions

- **Name.** "FlacToAiff" stops describing the app once WAV and ALAC are in. Rename before the first paid release, not after.
- **Trademarks.** rekordbox, CDJ and XDJ belong to AlphaTheta. Use them to describe compatibility ("prepares tracks for rekordbox"), never in the app name, icon or domain.
- **Licence.** Everything published so far stays MIT. If you want to sell, the usual pattern is: keep the core open, sell the signed build, and decide whether the rekordbox features go into a separate, closed part.
- **Strict vs. forgiving.** Refusing to save on any failed check is your best selling point; keep it for audio and core tags (finding 7), and be forgiving only where it costs nothing.
- **Mac App Store: later, if ever.** The sandbox needs bookmarks for every folder (finding 5), and launching `afconvert` from a sandboxed app needs testing. Direct sale plus notarization avoids all of that.

- **Support load.** A converter that touches a DJ's library before a gig gets urgent emails. Clear error texts (you already write good ones) and a "copy diagnostics" button save you time later.

Sources

- rekordbox-playlist-converter on PyPI
- rekordbox-edit: convert command
- Repository: SudoShinichi/flactoaiff, commit 2046626